Symbol	Meaning
D	Digit `[0–9]`
L	Lowercase letter `[a–z]`
U	Uppercase letter `[A–Z]`
WS	Whitespace `[ \t \r \n ]`
+ -	Plus or minus
.	Dot
e	Exponent e
other	Any other character

1. Integer Literal

Regex: `[+-]?[0-9]+`

Current State	Input	Next State
q0	+ or -	q1
q1	D	INT
INT	D	INT

Accepting State: `INT`

---

2. Floating-Point Literal

Regex: `[+-]?[0-9]+\.[0-9]+(e[+-]?[0-9]+)?`

Current State	Input	Next State
INT	.	q3
q3	D	q4
q4	D	q4
q4	e	q5
q4	ε	FP
q5	+ or -	q7
q7	D	FP
FP	D	FP

Accepting State: `FP`

3. Identifier

Regex: `[A-Z][a-z0-9_]*`

Current State	Input	Next State
q0	U	q8
q8	L, D, _	ID
ID	L, D, _	ID

Accepting State: `ID`

---

4. Boolean Literal: true

Regex: `true` (case-sensitive)

Current State	Input	Next State
Q0	'true'	Q14 (accept)

Accepting Token: `true`

---

5. Boolean Literal: false

Regex: `false` (case-sensitive)

Current State	Input	Next State
q0	'false'	q13

Accepting Token: `false`

---

6. Relational Operators

Regex: (`==` | `!=` | `<=` | `>=` | `<` | `>`)

a) Two-character operators

Current State	Input	Next State
q0	= | < | > | !	q15
q15	=	q16

Accepting State: `q16`

b) Single-character operators

Current State	Input	Next State
q0	< or >	q17

Accepting State: `q17`

---

7. Comments

Regex: `##[^\n]*`

Current State	Input	Next State
q0	#	q10
q10	#	q11
q11	any except #	Q12
Q12	any except #	Q12

Accepting State: `Q12 (single line comment)`

---

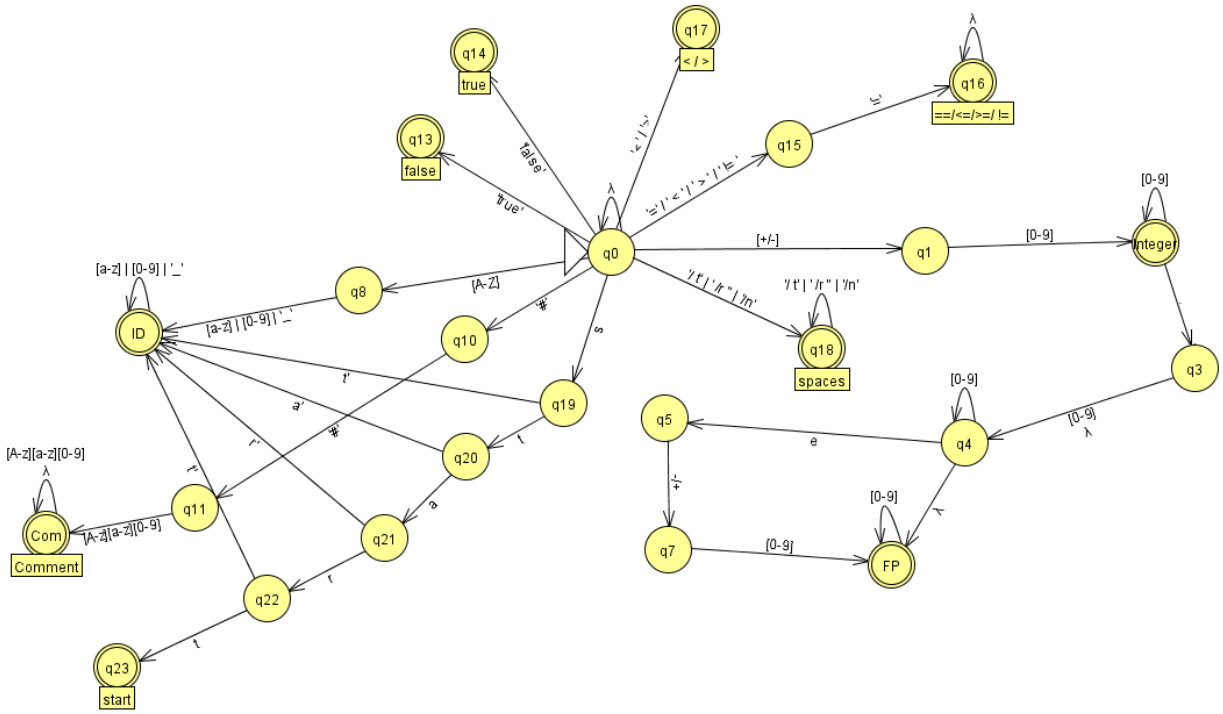
8. Whitespace

Regex: `[ \t \r \n ]+`

Current State	Input	Next State
q0	WS	Q18
Q18	WS	Q18 (SPACE)

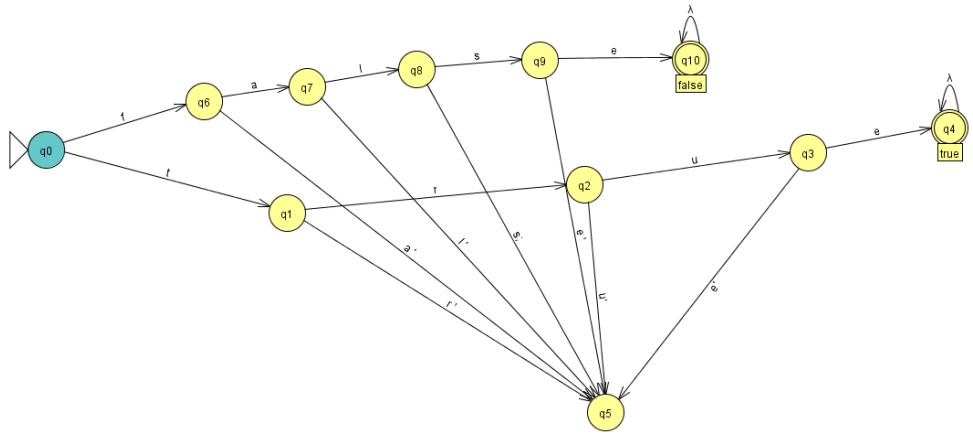
Accepting State: `SPACE` (Skipped)

Combined DFA:

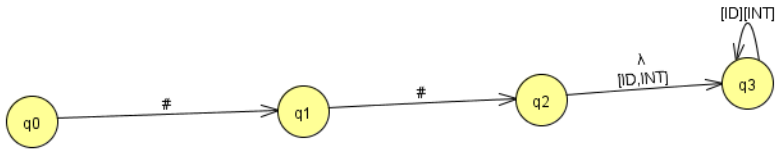


NFAS:

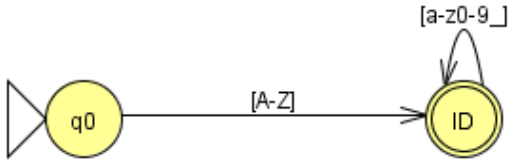
Boolean True/False



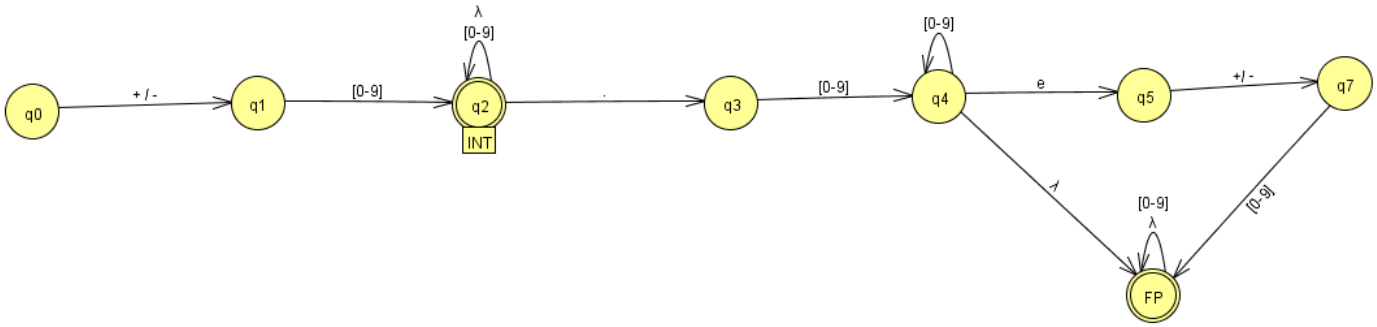
Single Line comment:



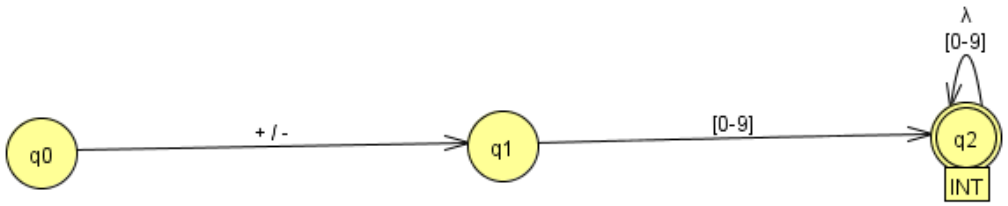
Identifier



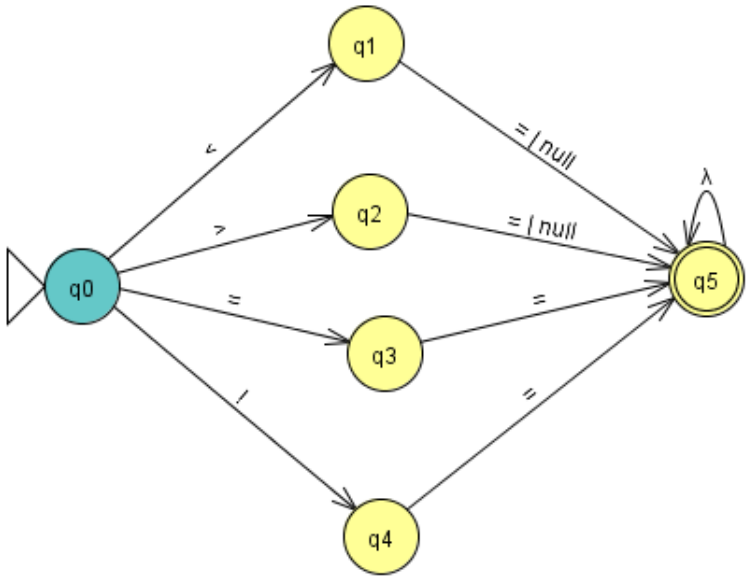
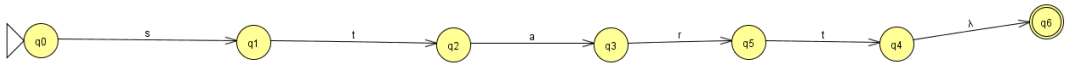
Floating point integers:



INT:



Comparison operators:



Start keyword: